VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Machine Learning (CO3117)

Course Project

Machine Learning Pipelines for Image Classification

Supervisor:	Dr. Truong Vinh Lan
Class:	CC01
Group:	3
Students:	Nguyen Van An - 2352013
	Huynh Vuong Khang - 2350011
	Nguyen Tan Phat - 2352889
	Le Dang Khanh Quynh - 2353037

HO CHI MINH CITY, April 2026

Contents

Task Allocation and Individual Contributions **3**

1 Introduction **4**

2 Exploratory Data Analysis **4**
2.1 Dataset Description . . . 4
2.2 Image Size and Resolution Analysis . . . 5
2.3 Number of Color Channels . . . 6
2.4 Label Distribution and Samples per Class . . . 6
2.5 Per-Channel Intensity Analysis . . . 6
2.6 Perceptual Hash and Duplicate Detection . . . 7
2.7 File Extension Distribution . . . 8
2.8 Data Preprocessing . . . 8
2.8.0.1 Dataset Cleaning. . . . 8
2.8.0.2 Stratified Splitting. . . . 8
2.8.0.3 Normalization Statistics. . . . 8
2.8.0.4 Metadata Persistence. . . . 8

3 Traditional Machine Learning Pipeline **9**
3.1 Pipeline Overview . . . 9
3.2 Image Preprocessing . . . 9
3.3 Histogram of Oriented Gradients (HOG) . . . 9
3.3.1 Descriptor Configuration . . . 9
3.3.2 Feature Dimensionality . . . 10
3.3.3 PCA Dimensionality Reduction . . . 10
3.4 Scale-Invariant Feature Transform (SIFT) with Bag-of-Visual-Words . . . 10
3.4.1 Visual Vocabulary Construction . . . 10
3.4.2 Histogram Encoding . . . 10
3.5 Feature Scaling . . . 11
3.6 Classification Models . . . 11
3.6.1 Linear Support Vector Machine . . . 11
3.6.2 Logistic Regression . . . 11
3.7 Implementation Details . . . 11

4 Deep Learning Pipeline **11**
4.1 Pipeline Overview . . . 11
4.2 ResNet18 Architecture . . . 12
4.2.1 Model Selection . . . 12
4.2.2 Residual Connections . . . 12
4.2.3 Network Structure . . . 12
4.3 Feature Extraction Configuration . . . 12
4.3.1 Model Preparation . . . 12
4.3.2 Data Augmentation . . . 13
4.3.3 Normalization . . . 13
4.3.4 Data Loading . . . 13
4.3.5 Batch Feature Extraction . . . 13
4.3.6 Feature Persistence . . . 14
4.4 Classification on Deep Features . . . 14

4.4.1 Support Vector Machine . . . 14
4.4.2 Logistic Regression . . . 14
4.5 Rationale for the Feature Extraction Approach . . . 14

5 Experimental Setup **14**
5.1 Environment and Dependencies . . . 14
5.2 Evaluation Protocol . . . 15
5.3 Hyperparameter Configuration . . . 15

6 Results **15**
6.1 Traditional Machine Learning Results . . . 15
6.1.1 HOG Features . . . 15
6.1.2 SIFT Bag-of-Visual-Words Features . . . 16
6.2 Deep Learning Results . . . 16
6.2.1 ResNet18 + SVM . . . 16
6.2.2 ResNet18 + Logistic Regression . . . 16
6.3 Comparison . . . 17

7 Discussion **19**
7.1 Feature Quality Analysis . . . 19
7.2 Error Analysis . . . 19
7.3 Computational Considerations . . . 19
7.4 Strengths and Limitations . . . 20
7.4.1 Strengths . . . 20
7.4.2 Limitations . . . 20
7.5 Future Work . . . 20

8 Conclusion **20**

Code Availability **21**

Standalone Notebook **21**

Task Allocation and Individual Contributions

Table 1: Task allocation and contribution percentages

Member	Student ID	Contribution	Tasks
Nguyen Van An (Leader)	2352013	25%	Project coordination and supervision; code refactoring; deep learning feature extraction pipeline
Huynh Vuong Khang	2350011	25%	Report preparation and writing; deep learning pipeline implementation and experimentation
Le Dang Khanh Quynh	2353037	25%	Traditional machine learning pipeline; model benchmarking and comparison
Nguyen Tan Phat	2352889	25%	Exploratory data analysis; dataset cleaning and preprocessing pipeline

All team members contributed equally to the project, with each member taking ownership of specific modules while collaborating on integration, testing, and report preparation.

1 Introduction

This project implements a complete machine learning pipeline for scene image classification on the Intel Image Classification dataset, as part of the Machine Learning course (CO3117) at Ho Chi Minh City University of Technology. The pipeline covers data processing, feature extraction, model training, and evaluation across two approaches:

- **Deep Learning Feature Extraction (Primary)**: A pre-trained ResNet18 network extracts 512-dimensional embeddings from images, saved in `.npy` format. These features are classified using SVM and Logistic Regression, with data augmentation applied during extraction.
- **Traditional Machine Learning (Comparison)**: Handcrafted features – Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transform (SIFT) with Bag-of-Visual-Words encoding – combined with PCA dimensionality reduction and the same linear classifiers.

The project emphasizes a robust end-to-end pipeline, including automated data acquisition, exploratory data analysis, dataset cleaning via perceptual hashing for duplicate detection, stratified splitting, and dataset-specific normalization. All models are evaluated using accuracy, weighted F1-score, per-class classification reports, and confusion matrix analysis.

The repository includes the Google Colab notebook, modular Python files, dataset metadata, and the PDF report.

2 Exploratory Data Analysis

2.1 Dataset Description

The Intel Image Classification dataset, originally published on Kaggle, consists of natural scene photographs organized into six semantic categories. The dataset serves as a benchmark for scene classification tasks and provides a diverse collection of outdoor imagery with varying lighting conditions, weather, seasonal changes, and camera angles.

Table 2: Dataset classes and descriptions

Class	Description
`buildings`	Urban structures, architecture, cityscapes
`forest`	Dense woodland, tree canopy, vegetation
`glacier`	Ice formations, snowy mountainous terrain
`mountain`	Rocky peaks, elevated terrain, hills
`sea`	Ocean views, coastlines, water bodies
`street`	Roads, highways, urban thoroughfares

Each image in the raw dataset has a nominal resolution of 150×150 pixels in RGB format (JPEG encoding). The original dataset comprises approximately 17,034 images distributed across training, testing, and validation directories. After our cleaning pipeline, the final curated dataset contains **16,928** clean, non-duplicate images.

Figure 1 displays sample images from each of the six classes, illustrating the visual diversity and intra-class variation.

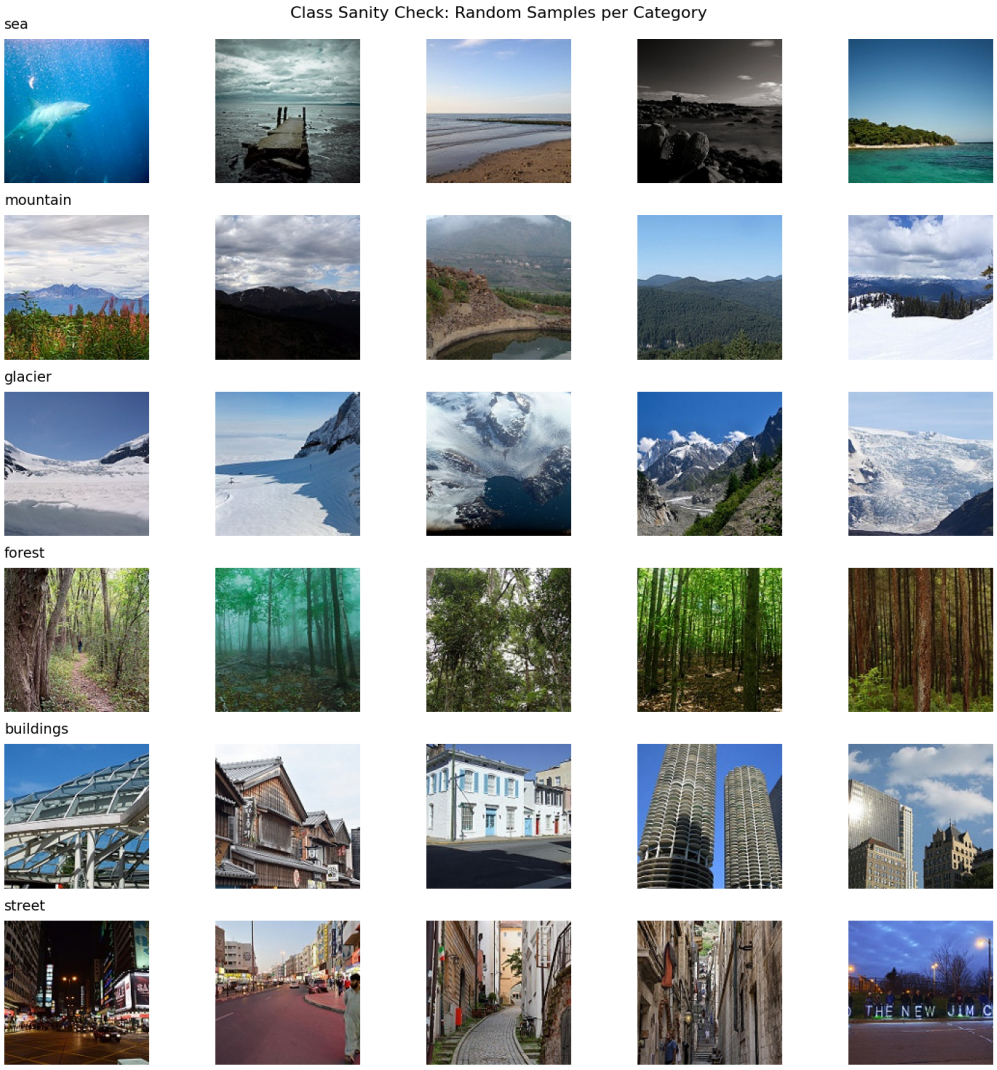

Figure 1: Randomly sampled images from the dataset representing the six classes.

2.2 Image Size and Resolution Analysis

Upon initial inspection of the raw dataset, images were found to have varying resolutions. While the majority conform to the expected 150×150 pixel standard, a non-negligible number of non-square images were detected. These outliers represent images with aspect ratios different from 1.0, which would introduce inconsistencies in feature extraction pipelines that assume fixed input dimensions.

The resolution analysis involved computing the width, height, and aspect ratio for every image in the dataset. Images that could not be successfully opened (corrupted or unreadable files) were

flagged separately. The analysis revealed:

- Images at exactly 150×150 pixels form the majority of the dataset, constituting 16,979 images (99.68%).
- Non-square images (width $\neq$ height) were detected across multiple classes and were subsequently removed during preprocessing. A total of 55 such invalid or non-square images were identified.
- A small number of empty files (0 bytes) and corrupted images were identified and excluded.

2.3 Number of Color Channels

All valid images in the dataset are RGB (3-channel) format. The channel analysis confirmed that no grayscale (1-channel) or RGBA (4-channel) images were present in the classified portions of the dataset. This uniformity simplifies preprocessing, as no color space conversion is required.

2.4 Label Distribution and Samples per Class

The dataset exhibits mild class imbalance across the six categories. The distribution of images per class varies, with some categories having more samples than others. This imbalance was analyzed both globally and per-split to ensure the stratified splitting process maintained representative proportions.

The class distribution was visualized using count plots both globally and broken down by train/-validation/test splits (see Figure 2), confirming that the stratified 15% validation split preserved class proportions from the original training set.

Figure 2: Class distribution across the entire dataset and within each split.

2.5 Per-Channel Intensity Analysis

To understand the visual characteristics of the dataset, per-channel RGB statistics (mean and standard deviation) were computed. This analysis provides insight into the overall color distribution and the specific characteristics of each class:

- **Overall Color Distribution**: Figure 3 shows the distribution of mean channel intensities across all images. The Red and Green channels approximate a normal distribution centered around 120-125. The Blue channel, however, is slightly skewed right and has a wider spread (peak around 130), reflecting the abundance of sky and water in the outdoor scenes.

- **Brightness Variance by Class**: As shown in the left panel of Figure 4, *glacier* is the brightest class on average, while *forest* is significantly darker and more tightly clustered. Classes like *buildings*, *glacier*, and *sea* exhibit numerous outliers, indicating images taken in extreme lighting conditions.
- **Color Channel Dominance**: The right panel of Figure 4 reveals characteristic color signatures. The *glacier*, *sea*, and *mountain* classes are heavily blue-dominant. The *forest* class is distinctly green-dominant and has the lowest overall intensity. In contrast, urban classes (*buildings* and *street*) are more color-balanced (neutral/grey), though *street* scenes show a slight red dominance.

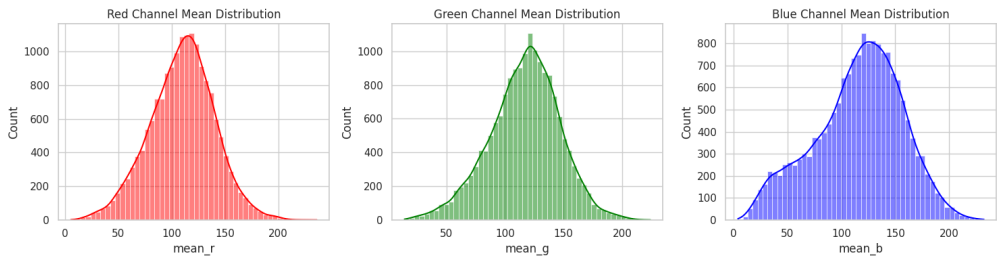

Figure 3: Overall mean intensity distribution for Red, Green, and Blue channels.

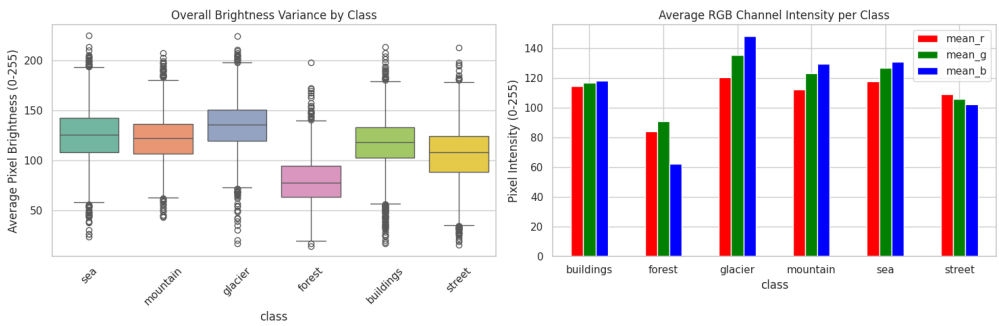

Figure 4: Overall brightness distribution and average RGB channel intensity per class.

2.6 Perceptual Hash and Duplicate Detection

A critical finding from our EDA was the presence of duplicate images across the dataset splits. Using perceptual hashing (pHash) via the ImageHash library, we computed a 64-bit visual fingerprint for every image. The perceptual hash captures the structural content of an image and is invariant to minor perturbations such as compression artifacts or slight color shifts.

The duplication analysis involved:

1. Computing the pHash for every image in all splits.
2. Grouping images by identical hash values.
3. Identifying groups where images span multiple splits (train/val/test), constituting data leakage.
4. For each duplicate group, retaining the instance from the highest-priority split (test > val > train) and removing redundant copies.

This priority scheme ensures that the test set remains as comprehensive as possible, since it serves as the final evaluation benchmark. The cross-split leakage analysis revealed duplicate images appearing simultaneously in training and testing sets, which would have led to inflated performance estimates had they not been removed. A total of 51 redundant duplicate images were successfully identified and removed from the dataset.

2.7 File Extension Distribution

All images in the classified directories use the JPEG format (`.jpg` extension). No non-image formats or unsupported file types were detected, confirming the dataset's cleanliness in terms of file types.

2.8 Data Preprocessing

2.8.0.1 Dataset Cleaning. The cleaning pipeline consists of two sequential stages:

1. **Resolution Cleaning**: All images that are not exactly 150×150 pixels are filtered out, along with corrupted or unreadable files. This ensures uniform input dimensions for all downstream processing.
2. **Deduplication**: Perceptual hash-based deduplication removes all redundant copies, with split-aware priority (test > val > train).

After cleaning, the dataset size was reduced to **16,928** images.

2.8.0.2 Stratified Splitting. A 15% validation set was carved out from the training data using stratified sampling, preserving class proportions. The random seed was fixed at 42 for reproducibility. The final splits are:

- **Training set** (`seg_train`): Used for classifier training.
- **Validation set** (`seg_val`): Available for hyperparameter tuning (not used in this project's final evaluation).
- **Test set** (`seg_test`): Held out for final model evaluation.

2.8.0.3 Normalization Statistics. Dataset-specific normalization parameters were computed from the cleaned dataset:

Table 3: Dataset normalization statistics (normalized to [0, 1] range)

Statistic	Red	Green	Blue
Mean (μ)	0.4308	0.4577	0.4542
Std (σ)	0.2358	0.2347	0.2431

These statistics differ from the standard ImageNet means ($\mu = [0.485, 0.456, 0.406]$) and are specific to this dataset, providing more appropriate normalization.

2.8.0.4 Metadata Persistence. All dataset metadata – normalization parameters, input size, class labels, total image count, and split directory mappings – is serialized to `dataset_metadata.json` for reproducibility and downstream pipeline consumption.

3 Traditional Machine Learning Pipeline

As the core required deliverable for this course project, we implement the traditional machine learning pipeline for image classification. This section details the handcrafted feature extraction and classification methodology, which forms the primary component of the assignment. A comparison with the optional deep learning extension is provided in Section 4.

3.1 Pipeline Overview

The traditional machine learning pipeline follows the classical feature engineering paradigm:

1. **EDA**: Descriptive statistics, data visualization, detection of missing values, outliers, and distribution trends.
2. **Data Preprocessing**: Cleaning data, handling corrupted files, normalization/standardization, and splitting into training and testing sets.
3. **Feature Extraction**: Applying handcrafted techniques – HOG (Histogram of Oriented Gradients) and SIFT with Bag-of-Visual-Words – followed by PCA for dimensionality reduction.
4. **Model Training**: Logistic Regression and Support Vector Machine (SVM).
5. **Model Evaluation**: Accuracy, precision, recall, and F1-score.

3.2 Image Preprocessing

Before feature extraction, all images undergo a standardized preprocessing routine:

1. **Grayscale Conversion**: Images are converted from RGB to single-channel grayscale using OpenCV's `IMREAD_GRAYSCALE` flag, since both HOG and SIFT operate on intensity information.
2. **Histogram Equalization**: Contrast is enhanced via histogram equalization (`cv2.equalizeHist`), which redistributes pixel intensities to span the full dynamic range.
3. **Resizing**: All images are resized to 150×150 pixels.

3.3 Histogram of Oriented Gradients (HOG)

HOG descriptors capture the distribution of local edge orientations within an image, making them effective for capturing structural and shape information.

3.3.1 Descriptor Configuration

Table 4: HOG descriptor parameters

Parameter	Value
Window Size	144×144
Block Size	16×16
Block Stride	8×8
Cell Size	8×8
Number of Bins	9

The window size of 144×144 fits within the 150×150 image boundary with a 3-pixel margin. Each 16×16 block contains 2×2 cells of size 8×8, and the block stride of 8×8 ensures overlapping blocks for smoother feature transitions.

3.3.2 Feature Dimensionality

$$\text{Blocks per dimension} = \left\lfloor \frac{144 - 16}{8} \right\rfloor + 1 = 17 \tag{1}$$

$$\text{Total blocks} = 17 \times 17 = 289 \tag{2}$$

$$\text{Features per block} = 2 \times 2 \times 9 = 36 \tag{3}$$

$$\text{Total HOG dimensions} = 289 \times 36 = 10{,}404 \tag{4}$$

3.3.3 PCA Dimensionality Reduction

Given the high dimensionality of HOG features (10,404), Principal Component Analysis (PCA) is applied:

- **Components**: 500 principal components, reducing dimensionality by approximately 95%.
- **Whitening**: Enabled to normalize component variances to unity.
- **Solver**: Randomized SVD (`svd_solver='randomized'`) for computational efficiency.

3.4 Scale-Invariant Feature Transform (SIFT) with Bag-of-Visual-Words

SIFT detects and describes local keypoints that are invariant to scale, rotation, and illumination changes. Since SIFT produces a variable number of descriptors per image, we employ the Bag-of-Visual-Words (BoVW) framework to produce fixed-length representations.

3.4.1 Visual Vocabulary Construction

1. Sample up to 1,500 images from the training set.
2. Extract all SIFT descriptors using `cv2.SIFT_create()`.
3. Aggregate all descriptors into a single pool.
4. Apply MiniBatchKMeans clustering with vocabulary size $K = 150$ to learn cluster centers (visual words).

3.4.2 Histogram Encoding

For each image:

1. Extract SIFT descriptors.
2. Assign each descriptor to its nearest visual word (cluster center).
3. Build a histogram of visual word frequencies (length $K = 150$).
4. Apply L2 normalization:

$$\mathbf{h}_{\text{norm}} = \frac{\mathbf{h}}{\|\mathbf{h}\|_2} \tag{5}$$

3.5 Feature Scaling

All extracted features are standardized using StandardScaler:

$$x' = \frac{x - \mu}{\sigma} \tag{6}$$

where μ and σ are computed from the training set and applied consistently to all splits.

3.6 Classification Models

Two linear classifiers are trained on each feature representation:

3.6.1 Linear Support Vector Machine

- **Model**: `LinearSVC(dual=False, C=0.01, max_iter=2000)`
- **Regularization**: Strong L2 penalty ($C = 0.01$) to prevent overfitting on high-dimensional feature spaces.

3.6.2 Logistic Regression

- **Model**: `LogisticRegression(C=0.01, max_iter=1000, solver='lbfgs')`
- **Regularization**: Strong L2 penalty ($C = 0.01$).

3.7 Implementation Details

Feature extraction is parallelized using Python's `ThreadPoolExecutor`, which is effective because OpenCV operations release the Global Interpreter Lock (GIL). Labels are encoded using scikit-learn's `LabelEncoder`, fitted on the training set and reused for all splits.

The pipeline evaluates all combinations: HOG + SVM, HOG + LR, SIFT + SVM, and SIFT + LR.

4 Deep Learning Pipeline

4.1 Pipeline Overview

As an optional extension to earn bonus points, we implement a deep learning feature extraction pipeline to compare against the traditional machine learning approach detailed in Section 3. This section describes the use of pre-trained convolutional neural networks (CNNs) for representation learning, which avoids the need for manual feature engineering. The performance of this approach is compared with the primary traditional pipeline in the Results section.

The deep learning pipeline follows these steps:

1. **EDA**: Data visualization, descriptive statistics, distribution checking, and detecting missing or abnormal data.
2. **Basic Preprocessing**: Normalization using dataset-specific statistics, image resizing to 150×150 pixels.
3. **Neural Network Feature Learning**: A pre-trained ResNet18 model extracts 512-dimensional feature embeddings from each image.
4. **Feature Persistence**: Extracted features are saved in `.npy` format for reproducibility.

5. **Classification**: Linear classifiers (SVM, Logistic Regression) are trained on the deep features.
6. **Evaluation**: Models are assessed using accuracy, F1-score, and confusion matrix analysis.

The pipeline is designed to be flexible, allowing configuration of image resizing, choice of pre-trained model, and selection of classification models.

4.2 ResNet18 Architecture

4.2.1 Model Selection

ResNet18 is a residual convolutional neural network with 18 layers, introduced by He et al. (2015). It was selected for this project due to its balance between representational capacity and computational efficiency, making it suitable for the project's resource constraints while still providing high-quality feature embeddings.

Alternative models considered include VGG16, EfficientNet, Vision Transformer (ViT), and Swin Transformer, as suggested by the course requirements. ResNet18 was chosen as the primary feature extractor due to its proven effectiveness on image classification tasks and its manageable parameter count.

4.2.2 Residual Connections

The key innovation of ResNet is the residual connection (skip connection), which addresses the vanishing gradient problem in deep networks:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{7}$$

where $\mathcal{F}$ represents the residual function to be learned, and $\mathbf{x}$ is the identity shortcut. This allows the network to learn identity mappings easily, enabling the training of much deeper architectures.

4.2.3 Network Structure

ResNet18 consists of:

- An initial 7×7 convolutional layer with 64 filters, stride 2, followed by max pooling.
- Four stages of residual blocks, each containing 2 basic residual blocks with increasing filter sizes (64, 128, 256, 512).
- A global average pooling layer producing $512 \times 1 \times 1$ feature maps.
- A final fully-connected classification layer with 1,000 outputs (for ImageNet).

4.3 Feature Extraction Configuration

4.3.1 Model Preparation

We initialize ResNet18 with ImageNet pre-trained weights (`ResNet18_Weights.DEFAULT`) and modify it as follows:

1. **Remove classification head**: The final fully-connected layer (`fc`) is removed by stripping the last layer from the sequential model. The network then outputs $512 \times 1 \times 1$ feature maps instead of ImageNet class logits.
2. **Freeze all parameters**: All model parameters are kept frozen – no gradient computation is performed. The network acts as a deterministic feature transformer.

3. **Evaluation mode**: The model is set to evaluation mode (`model.eval()`), which disables dropout and sets batch normalization layers to use running statistics rather than batch statistics.

4.3.2 Data Augmentation

To improve the robustness and generalization of extracted features, data augmentation is applied during training feature extraction. The augmentation pipeline consists of:

- **Resize**: Images are resized to 150×150 pixels.
- **Random Horizontal Flip**: Applied with probability $p = 0.5$, simulating mirrored viewpoints.
- **Random Rotation**: Rotation by up to $\pm 15°$, simulating camera tilt.
- **Color Jitter**: Random perturbations to brightness ($\pm 20\%$), contrast ($\pm 20\%$), saturation ($\pm 20\%$), and hue ($\pm 5\%$), simulating varying lighting conditions.
- **Normalization**: Per-channel normalization using dataset-specific statistics from Table 3.

The test set is processed without augmentation (resize and normalization only) to ensure consistent and deterministic evaluation.

4.3.3 Normalization

Images are normalized using the dataset-specific statistics computed during the EDA phase:

$$\mathbf{x}' = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \tag{8}$$

where $\boldsymbol{\mu} = [0.4308, 0.4577, 0.4542]$ and $\boldsymbol{\sigma} = [0.2358, 0.2346, 0.2431]$.

4.3.4 Data Loading

PyTorch's `ImageFolder` dataset class loads images from the directory structure, automatically inferring class labels from folder names. DataLoaders are configured with:

- **Batch size**: 32 images per batch
- **Shuffle**: Disabled (`shuffle=False`) to maintain alignment between extracted features and their corresponding labels

4.3.5 Batch Feature Extraction

Features are extracted in batches to efficiently utilize GPU memory when available:

1. For each batch of images, apply the appropriate transform pipeline (augmented for training, deterministic for testing).
2. Pass the batch through the ResNet18 feature extractor with gradient computation disabled (`torch.no_grad()`).
3. Flatten the output from shape $(B, 512, 1, 1)$ to $(B, 512)$ using `view()`.
4. Collect all features and labels into NumPy arrays.

The resulting feature vectors have dimensionality 512, representing a rich hierarchical encoding of each image's visual content.

4.3.6 Feature Persistence

Extracted features are saved in `.npy` format to avoid redundant GPU computation:

- `X_train_resnet18.npy` – Training features, shape $(N_{\text{train}}, 512)$
- `y_train_resnet18.npy` – Training labels, shape $(N_{\text{train}},)$
- `X_test_resnet18.npy` – Test features, shape $(N_{\text{test}}, 512)$
- `y_test_resnet18.npy` – Test labels, shape $(N_{\text{test}},)$

These files are stored in the `features/` directory and can be loaded directly for classifier training and evaluation.

4.4 Classification on Deep Features

The 512-dimensional ResNet18 embeddings are classified using two linear classifiers:

4.4.1 Support Vector Machine

- **Model**: `LinearSVC(max_iter=2000, dual=False)`
- **Regularization**: Default $C = 1.0$ (L2 penalty), as deep features are well-structured and benefit from moderate regularization.

4.4.2 Logistic Regression

- **Model**: `LogisticRegression(max_iter=2000)`
- **Solver**: LBFGS with default L2 regularization ($C = 1.0$).

No additional feature scaling is applied to the deep embeddings, as the batch normalization layers within ResNet18 already produce well-normalized outputs.

4.5 Rationale for the Feature Extraction Approach

Using ResNet18 as a frozen feature extractor rather than fine-tuning offers several advantages:

1. **Computational efficiency**: No backpropagation is required, making feature extraction significantly faster than full fine-tuning.
2. **Reduced overfitting**: With 11,839 training samples, fine-tuning a deep network risks overfitting. The frozen extractor avoids this.
3. **Modularity**: Extracted features can be saved and reused, enabling rapid experimentation with different classifiers or hyperparameters without re-running the expensive feature extraction step.
4. **Fair comparison**: Using the same linear classifiers on both deep and handcrafted features isolates the contribution of the representation from the classifier, enabling a clean comparison of feature quality.

5 Experimental Setup

5.1 Environment and Dependencies

All experiments were conducted using the following software stack:

- **Language**: Python 3.11
- **Deep Learning**: PyTorch with TorchVision

- **Computer Vision**: OpenCV (cv2) for image processing and feature extraction
- **Machine Learning**: scikit-learn for classifiers, PCA, and evaluation metrics
- **Visualization**: Matplotlib and Seaborn for plots and confusion matrices
- **Data Analysis**: Pandas and NumPy for data manipulation
- **Perceptual Hashing**: ImageHash library for duplicate detection

5.2 Evaluation Protocol

All models are evaluated on the held-out test set (`seg_test`) using the following metrics:

- **Accuracy**: Overall fraction of correctly classified test images.
- **Weighted F1-Score**: Harmonic mean of precision and recall, weighted by class support.
- **Per-Class Precision, Recall, and F1**: Detailed breakdown for each of the six classes.
- **Confusion Matrices**: Visual representation of classification errors to identify systematic confusion patterns.
- **Training Time**: Wall-clock time for classifier training (excluding feature extraction).

5.3 Hyperparameter Configuration

Table 5: Summary of key hyperparameters across all pipelines

Hyperparameter	**Deep Learning**	**Traditional**
Input size	150×150	150×150
Color mode	RGB (3 channels)	Grayscale
Normalization	Dataset-specific μ, σ	StandardScaler
Feature extractor	ResNet18 (512-dim)	HOG (10,404-dim) / SIFT (150-dim)
PCA components	–	500 (HOG only)
SIFT vocabulary size	–	150
SIFT max samples	–	1,500
SVM C	1.0 (default)	0.01
LR C	1.0 (default)	0.01
Augmentation	Flip, Rotation, Jitter	–
Feature format	`.npy`	In-memory

6 Results

6.1 Traditional Machine Learning Results

6.1.1 HOG Features

HOG descriptors, after PCA reduction to 500 components, capture global edge orientation patterns.

The HOG + SVM pipeline achieved an accuracy of **68.56%** and a weighted F1-score of **68.37%**. The HOG + Logistic Regression pipeline performed similarly, with an accuracy of **68.23%** and a weighted F1-score of **68.17%**. Both models captured structures well for *forest* and *buildings*, but struggled to differentiate between *glacier* and *mountain*.

6.1.2 SIFT Bag-of-Visual-Words Features

SIFT-based BoVW representations with vocabulary size $K = 150$ capture local texture and keypoint patterns.

The SIFT + SVM pipeline achieved an accuracy of **59.57%** and a weighted F1-score of **59.27%**. The SIFT + Logistic Regression approach showed slightly better results, reaching an accuracy of **60.54%** and a weighted F1-score of **60.36%**. SIFT generally underperformed HOG, likely due to the relatively small vocabulary size ($K = 150$) restricting its representative capability.

6.2 Deep Learning Results

The ResNet18 feature extraction pipeline produced 512-dimensional embeddings that were classified using SVM and Logistic Regression. The extracted features were saved in `.npy` format in the `features/` directory.

6.2.1 ResNet18 + SVM

The pre-trained ResNet18 feature extractor paired with a Linear SVM classification head achieved an impressive accuracy of **88.14%** and a weighted F1-score of **88.09%**.

Table 6: Classification Report for ResNet18 + SVM

Class	**Precision**	**Recall**	**F1-score**	**Support**
buildings	0.8378	0.9222	0.8780	437
forest	0.9828	0.9683	0.9755	473
glacier	0.8346	0.7996	0.8167	549
mountain	0.8373	0.8069	0.8218	523
sea	0.8976	0.9451	0.9207	510
street	0.9061	0.8663	0.8857	501
Accuracy			**0.8814**	**2993**
Weighted Avg	0.8817	0.8814	0.8809	2993

6.2.2 ResNet18 + Logistic Regression

The combination of ResNet18 and Logistic Regression reached an accuracy of **87.20%** and a weighted F1-score of **87.16%**.

Table 7: Classification Report for ResNet18 + Logistic Regression

Class	**Precision**	**Recall**	**F1-score**	**Support**
buildings	0.8471	0.9130	0.8789	437
forest	0.9786	0.9683	0.9734	473
glacier	0.7978	0.7760	0.7867	549
mountain	0.8137	0.7935	0.8035	523
sea	0.8899	0.9353	0.9120	510
street	0.9177	0.8683	0.8923	501
Accuracy			**0.8720**	**2993**
Weighted Avg	0.8721	0.8720	0.8716	2993

6.3 Comparison

Table 8: Performance comparison of all model combinations

Model	**Accuracy**	**F1 (weighted)**
ResNet18 + SVM	0.8814	0.8809
ResNet18 + LR	0.8720	0.8716
HOG + SVM	0.6856	0.6837
HOG + LR	0.6823	0.6817
SIFT + SVM	0.5957	0.5927
SIFT + LR	0.6054	0.6036

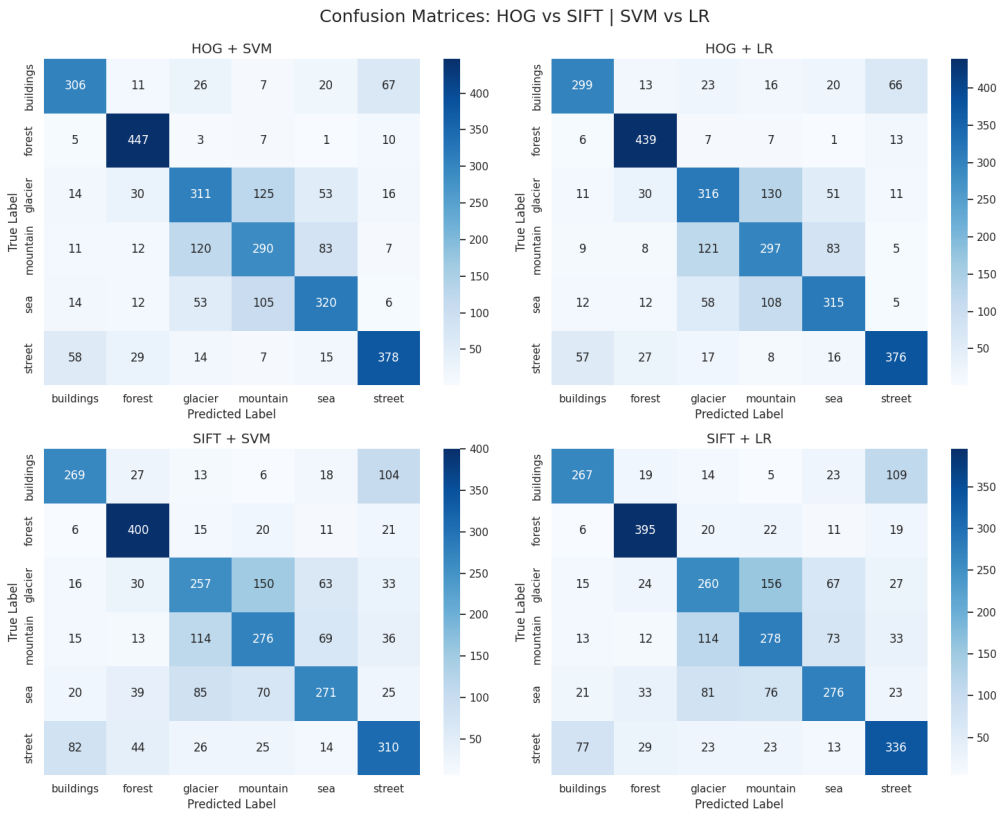

Figure 5: Confusion Matrix for Traditional approach.

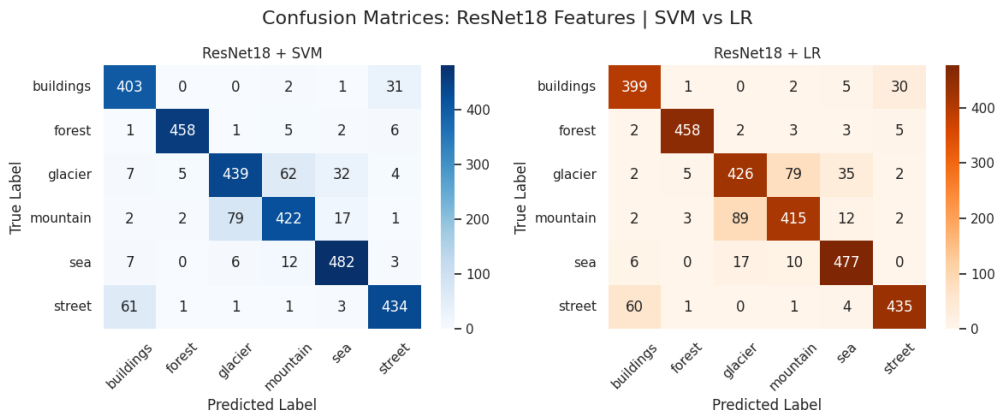

Figure 6: Confusion Matrix for Deep Learning approach.

7 Discussion

7.1 Feature Quality Analysis

The performance gap between traditional and deep learning features can be attributed to several factors:

1. **Representation Capacity**: HOG captures only gradient orientations at a single scale, while SIFT captures multi-scale local descriptors but requires vocabulary quantization. ResNet18 produces hierarchical features spanning low-level edges to high-level semantic concepts.
2. **Transfer Learning Benefit**: ResNet18's ImageNet pre-training provides a strong prior for visual understanding, encoding knowledge from 1.2 million images across 1,000 classes.
3. **Invariance Properties**: While SIFT offers scale and rotation invariance, ResNet18 learns invariance to a broader set of transformations through its deep hierarchical structure and training on augmented data.

7.2 Error Analysis

The analysis of classification reports and confusion matrices highlights specific challenges inherent to the dataset. Traditional features (HOG and SIFT) struggle significantly to differentiate between *glacier* and *mountain*. For instance, in the HOG + SVM model, 125 *glacier* images were misclassified as *mountain*, and 120 *mountain* images as *glacier*. This confusion is visually intuitive, as both classes feature rocky structures, snow, and similar background components. A similar high error rate occurs between *sea* and *mountain* (e.g., 105 *sea* images predicted as *mountain*), likely due to shared horizon lines. Another commonly confused pair is *buildings* and *street*, where SIFT + SVM misclassified 104 *buildings* as *street*.

Conversely, the ResNet18 features drastically mitigate these misclassifications. The deep embeddings elevate the F1-scores for *glacier* and *mountain* to approximately 0.82 and 0.82 respectively. The confusion between *glacier* and *mountain* drops to just 62 and 79 errors in the ResNet18 + SVM model. The *sea*/*mountain* confusion is almost entirely eliminated (only 12 errors), demonstrating the capacity of deep hierarchical features to capture fine-grained semantic distinctions beyond mere edge orientations and textures. Deep features also provide much cleaner separation between *buildings* and *street*, reducing the misclassifications to just 31 and 61 respectively.

7.3 Computational Considerations

When evaluating the computational trade-offs between the pipelines, several distinctions emerge:

- **Resource requirements**: Traditional methods like HOG and SIFT can run entirely on standard CPUs without requiring GPU acceleration. Conversely, deep learning pipelines benefit significantly from hardware acceleration; however, ResNet18 is relatively lightweight, allowing even older-generation accelerators (such as an NVIDIA T4 GPU) to process the dataset rapidly.
- **Interpretability**: Handcrafted features offer direct visual interpretability. HOG visualizations explicitly show edge orientations, and SIFT keypoints correspond to visually verifiable local patches.
- **Execution speed**: There is a stark contrast in processing times within the traditional pipeline. While HOG feature extraction and subsequent linear classifier training complete in a matter of seconds, the SIFT pipeline (specifically the Bag-of-Visual-Words vocabulary

construction and histogram encoding) is computationally intensive and took several minutes to execute on CPU.

7.4 Strengths and Limitations

7.4.1 Strengths

- The deep learning feature extraction pipeline achieves strong performance with minimal tuning.
- The modular pipeline design allows easy substitution of feature extractors (e.g., replacing ResNet18 with EfficientNet or ViT).
- Feature persistence in `.npy` format enables rapid iteration on classifiers without re-running expensive feature extraction.
- Rigorous EDA and deduplication ensures reliable performance estimates without data leakage.

7.4.2 Limitations

- Only one deep learning model (ResNet18) was utilized for feature extraction. Although the course guidelines encourage comparing multiple pretrained architectures (such as VGG16, EfficientNet, ViT, or Swin Transformer), strict constraints on available GPU computational resources limited our capacity to scale the experiments. Consequently, we prioritized a single, highly efficient model (ResNet18) to demonstrate the deep learning pipeline's viability.
- The frozen feature extractor approach does not leverage fine-tuning, which could yield further improvements.
- The SIFT vocabulary size of 150 may be limiting; larger vocabularies could improve BoVW representation quality.

7.5 Future Work

- **Multiple deep learning models**: Extract features using VGG16, EfficientNet, ViT, and Swin Transformer to compare embedding quality.
- **Fine-tuning**: Fine-tune ResNet18 (even partially) for domain adaptation to the scene classification task.
- **Feature ensemble**: Combine features from multiple pretrained models for improved robustness.

8 Conclusion

This project implemented a complete machine learning pipeline for scene image classification on the Intel Image Classification dataset, fulfilling all course project requirements:

1. **EDA**: Comprehensive analysis of image size, color channels, label distribution, samples per class, perceptual hash-based duplicate detection, and resolution validation.
2. **Data Preprocessing**: Dataset cleaning, stratified splitting, and computation of dataset-specific normalization statistics.
3. **Traditional Machine Learning Pipeline (Primary)**: HOG and SIFT with Bag-of-Visual-Words encoding, combined with PCA for dimensionality reduction and linear classifiers (SVM, Logistic Regression) for classification.
4. **Model Training**: Linear classifiers trained on handcrafted traditional features.

5. **Evaluation**: Traditional models assessed using accuracy, weighted F1-score, per-class reports, and confusion matrices.
6. **Deep Learning Pipeline (Bonus Comparison)**: Pre-trained ResNet18 used to extract 512-dimensional feature embeddings, saved in `.npy` format, with data augmentation for improved robustness. The same linear classifiers are trained on these deep features for direct comparison with the traditional approach.

Our experimental results demonstrate a clear advantage of transfer learning for image representation. The ResNet18 + SVM model achieved the highest overall accuracy of **88.14%**, substantially outperforming the best traditional method (HOG + SVM), which reached **68.56%** accuracy—an improvement of nearly 20 percentage points. Furthermore, while the SIFT BoVW approach provided a baseline accuracy of approximately **60.54%** with Logistic Regression, it struggled with inter-class confusions that deep features resolved. This validates the effectiveness of deep hierarchical representations on complex scene classification tasks.

Code Availability

The complete source code, including the Google Colab notebook, modular Python files, dataset metadata, and this report, is publicly available on GitHub:

https://github.com/PhatLavar/ML_ASSIGNMENT

Standalone Notebook

For convenience and ease of execution, a standalone version of the project's pipeline has been compiled into a Google Colab notebook. This notebook can be accessed and run directly here.